

Saga、服务编排、时间溯源

一 saga

Saga 是一种分布式事务处理模式，主要用于在微服务架构中确保数据一致性。传统的分布式事务（如两阶段提交）在微服务环境中可能并不适用，因为它们通常会导致系统的耦合度过高、性能下降以及可用性问题。Saga 模式通过将一个全局事务拆分成一系列本地事务，每个本地事务由一个微服务来完成，并且每个本地事务都能够独立地提交和回滚。

Saga 模式有两种常见的实现方式：补偿事务（Compensating Transactions）和基于事件的协调（Event Choreography）。

1 补偿事务

在补偿事务中，每个本地事务都有一个对应的补偿事务，用于在出现问题时回滚之前的操作。这些本地事务按照一定的顺序执行，如果其中某个事务失败，Saga 会按相反的顺序依次执行这些事务的补偿操作，以撤销之前的成功操作。

场景：旅行预订系统

假设我们有一个旅行预订系统，用户可以在同一个订单中预订航班、酒店和租车服务。每个服务都是由不同的微服务处理的。整个预订过程涉及多个步骤，每个步骤都是一个本地事务：如果某一步失败，我们需要回滚之前的步骤。

1. 创建预订（航班、酒店、租车）
2. 扣减库存（航班座位、酒店房间、租车车辆）
3. 处理支付

流程：

1. 用户提交预订请求。
2. 创建预订：
 - 航班服务创建航班预订记录。

- 酒店服务创建酒店预订记录。
- 租车服务创建租车预订记录。

3. 扣减库存:

- 航班服务扣减座位库存。
- 酒店服务扣减房间库存。
- 租车服务扣减车辆库存。

4. 处理支付:

- 支付服务处理支付。

补偿流程:

如果支付处理失败，需要执行以下补偿事务:

1. 航班服务回滚座位库存。
2. 酒店服务回滚房间库存。
3. 租车服务回滚车辆库存。
4. 取消所有的预订记录。

2、基于事件的协调场景

场景：在线教育平台:

假设我们有一个在线教育平台，用户可以购买课程。购买课程后，系统需要依次完成以下任务：生成订单、扣减课程库存、处理支付、发送确认邮件。

流程

1. 用户提交订单:
 - 用户在平台上选择课程并提交订单请求。
2. 订单服务处理:
 - 订单服务生成订单，并发布“订单已创建”事件。

3. 库存服务处理:

- 库存服务接收到“订单已创建”事件后，扣减课程库存，并发布“库存已扣减”事件。

4. 支付服务处理:

- 支付服务接收到“库存已扣减”事件后，处理支付，并发布“支付已完成”事件。

5. 邮件服务处理:

- 邮件服务接收到“支付已完成”事件后，发送确认邮件给用户。

补偿流程:

如果支付处理失败，支付服务发布“支付失败”事件，以下微服务需要执行相应的补偿操作:

1. 库存服务接收到“支付失败”事件后，回滚课程库存。
2. 订单服务接收到“支付失败”事件后，取消订单。

二、服务编排

服务编排是一种集中控制的服务调用方式，通过一个中央控制器来管理和协调多个服务的调用，以完成一个复杂的业务流程。服务编排的特点是业务流程的逻辑集中在一个地方，这使得整个流程的管理和维护更加容易。

1 工作原理

服务编排主要由一个编排器（Orchestrator）来执行，它负责按顺序调用各个微服务，并处理每个服务的响应。编排器包含了业务流程的所有逻辑，确保各个服务按照指定的步骤执行。

2、优缺点

(1) 优点

- **集中控制：**业务流程逻辑集中在编排器中，易于管理和维护。
- **明确的执行顺序：**编排器确保服务按照预定的顺序执行，减少了服务间的复杂依赖。
- **简化了服务的开发：**各微服务只需关注自己的业务逻辑，不需要了解整个业务流程。

(2) 缺点

- **单点故障：**编排器是单点，如果编排器故障，整个流程会受到影响。
- **耦合度较高：**业务流程的逻辑集中在编排器中，可能导致系统的灵活性和扩展性降低。
- **性能瓶颈：**所有服务调用都经过编排器，可能成为性能瓶颈。

3、场景：在线医疗预约系统

在一个在线医疗预约系统中，用户可以预约医生、支付预约费用并接收确认信息。这个过程需要调用多个微服务，包括用户服务、医生服务、支付服务和通知服务。我们可以使用服务编排来协调这些服务的调用。

流程：

1. 用户在平台上选择医生和时间并提交预约请求。
2. 编排器接收预约请求，调用用户服务验证用户信息。
3. 用户服务验证成功后，编排器调用医生服务检查医生的可用时间。
4. 医生服务确认可用时间后，编排器调用支付服务处理预约费用支付。
5. 支付服务处理支付成功后，编排器调用通知服务发送预约确认信息给用户和医生。

6. 通知服务发送确认信息后，编排器向用户返回预约完成的通知。

补偿事务：

如果支付处理失败，编排器需要撤销之前的操作：

- 编排器调用医生服务释放医生的预约时间。
- 编排器调用用户服务记录预约失败信息。

三、事件溯源

事件溯源 (Event Sourcing) 是一种数据管理模式，其中所有的状态变化（事件）都被保存下来，并且系统的当前状态通过重放这些事件来构建。这与传统的 CRUD（创建、读取、更新、删除）操作不同，事件溯源关注的是记录引起状态变化的所有事件，而不是直接存储最新的状态。

1 核心概念

1. **事件 (Event)**：表示系统状态变化的不可变记录。例如，在电子商务系统中，事件可以是“订单创建”、“订单支付”、“订单发货”等。
2. **事件存储 (Event Store)**：一种专门的数据库，用于持久化存储所有事件。事件存储不仅保存事件的详细信息，还保存事件发生的顺序。
3. **聚合 (Aggregate)**：一种聚集相关事件的逻辑单元，通常对应于领域模型中的实体或聚合根。聚合可以从事件存储中重放事件来恢复其当前状态。
4. **事件重放 (Event Replay)**：将存储的事件按顺序重放，来重新构建系统或聚合的当前状态。

2 优缺点

优点：

- **完全可追溯**：每个事件都被记录下来，可以回溯到任何一个时刻的状态。
- **审计和调试**：提供了详细的操作日志，便于审计和调试。

- **系统恢复：**通过重放事件，可以在灾难恢复时重建系统状态。
- **事件驱动架构：**与 CQRS（命令查询职责分离）模式结合，提供了良好的扩展性和可维护性。

缺点：

- **存储开销：**需要存储所有历史事件，可能会带来较大的存储需求。
- **复杂性：**实现和维护事件溯源系统的复杂性较高，尤其是在处理事件的版本控制和数据一致性时。
- **性能问题：**重放大量事件可能会影响性能，需要额外的优化措施（如快照）。

3 场景：银行账户管理系统

在一个银行账户管理系统中，用户可以进行存款、取款和转账操作。使用事件溯源来管理账户状态的变化，每个操作都被记录为一个事件。

事件类型：

存款事件 (Deposit Event)

取款事件 (Withdrawal Event)

转账事件 (Transfer Event)

流程：

存款操作：

用户向账户存款。

生成“存款事件”，记录存款金额和时间。

将事件保存到事件存储中。

取款操作：

用户从账户取款。

生成“取款事件”，记录取款金额和时间。

将事件保存到事件存储中。

转账操作：

用户从一个账户向另一个账户转账。

生成“转账事件”，记录转账金额、时间和目标账户。

将事件保存到事件存储中。

恢复账户状态：

从事件存储中获取账户的所有事件。

按时间顺序重放这些事件，计算账户的当前余额。